

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

playlist.m3u8

init.mp4

000001.m4s

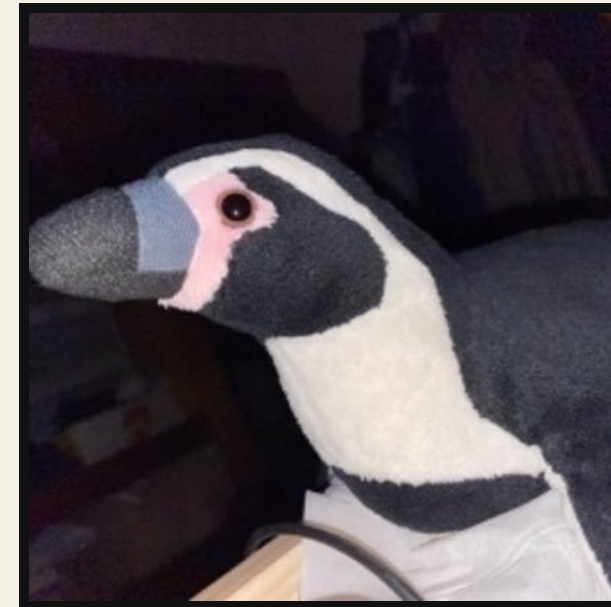
— ABOUT ME

Hikaru Sato

@SatoHikaruDev

主に iOS / Android / Ruby on Rails のアプリ開発をしています

[MomentNow](#) という「今この瞬間」の動画をHLS
で配信し、
URLで共有できる iOSアプリ を個人開発してます



今回は MomentNow での端末内HLS生成の仕組みや実装を分解します

話すこと

WHY なぜ端末内HLSを作ろうと思ったのか

01 HLSがライブになる仕組み

02 端末内でHLSを生成

03 約2秒の動画断片を生成

04 playlistの更新

05 配信の完了

撮影後のアップロードを待たずに、動画をURLで共有したい

HLSなら、撮影中から小さくアップロードし、撮影後URLで再生できる

1ファイル

`recording.mov`

撮影後にまとめてupload

↓ 待ち時間をなくす

HLS

再生設定 短い動画 1

短い動画 2

撮影中から順次upload → 同じURLで共有

macOS向けAppleサンプルが とても参考になった

2025.03

AIで最初の試作

HLSとしての
実用まで至らず



Apple公式

生成手順を確認

Writer設定
時刻補正
Dataの受け取り



iPhone local

HLS生成に成功

カメラ・マイク
短い動画を生成



MomentNow

ライブ配信へ発展

撮影中からupload
playlistを更新

端末内生成で変換サーバーを減らせると分かり、MomentNowの開発へ進んだ

01

HLSで「ライブ」になる仕組み

短い動画ファイル + 更新され続けるplaylist

HLSは、短い動画と 更新されるplaylistをHTTPで配る

```
#EXTM3U
#EXT-X-MAP: init.mp4
#EXTINF: 2.000
seg/000001.m4s
#EXTINF: 2.000
seg/000002.m4s
```



EXT-X-MAP

初期化segmentの場所

EXTINF

次のsegmentの長さ

URI

次に取得するファイル

— THREE FILE TYPES

HLSライブ配信を構成する、3種類のファイル

M3U8	MP4	M4S
playlist.m3u8 更新される再生順	init.mp4 再生を始めるための設定	000001.m4s 2秒前後の映像と音声

init.mp4（再生準備） + m4s（約2秒） + playlist（順番） = 最初の再生

fragmented MP4 (fMP4) は 「設定」と「映像・音声の断片」を分ける

1 配信に1つ

init.mp4

Playerが再生を始めるための設定

+

約2秒ごと

000001.m4s

実際の映像と音声

=

playlistが結ぶ

再生可能

Playerはこの組み合わせを取得

— WHY IT IS LIVE

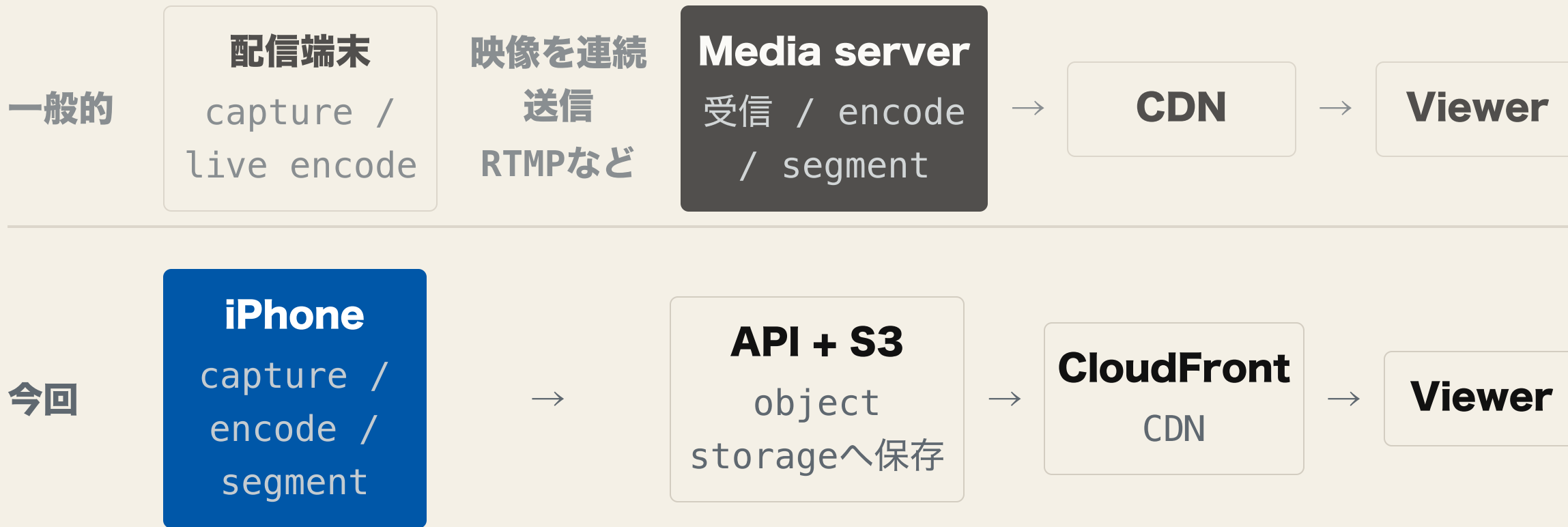
playlistが増えるたび、 Playerは次のsegmentを取得する

t = 0	t = 2s	t = 4s	stop
playlistだけ segmentなし	+ seg 1 追記	+ seg 2 追記	#EXT-X-ENDLIST もう増えない

Player (AVPlayer / Safariなどの再生エンジン) は同じplaylistを繰り返しGETする

— FROM HLS TO THIS TALK

HLSを作る場所を、サーバーからiPhoneへ



今回なくすのは、映像を受信し続けて変換するサーバー

— THIS TALK'S IMPLEMENTATION

iPhoneが生成したHLSを 撮影中からS3へ公開する

映像と音声を受け
取る

Camera + Mic



約2秒に分ける

AVAssetWriter



保存先へ置く

HTTP PUT



再生順を公開する

playlist更新

— LIVE DEMO · PUBLIC SAMPLE

公開サンプルアプリを動かす

HLSSample

Camera Preview

• 配信中



HTTP
PUT

start

PUT init.mp4

+2s

PUT seg 1

after PUT

PUT playlist

Mac HTTP Server / Viewer

LIVE LOCAL

playlist.m3u8を追従再生

github.com/HikaruSato/iosdc2026HLSSample

Server は「PUTされたファイルを残すだけ」

Mac

```
$ python3 server/server.py
```

Viewer: <http://localhost:8080>

iPhone server URL:

<http://192.168.x.x:8080>

PUT init.mp4 / m4s / m3u8

DISK server/data/streams/...

GET 同じファイルをViewerへ
返す

「配信開始」で、iPhoneがHLS生成を始める

● 配信開始

→

Camera + Mic
video frame /
audio block
(CMSampleBuffer)

→

AVAssetWriter
圧縮してHLS用Dataへ
分割

elapsed 0.0 → segment 0 →

最初に届くのは、1回だけのinit.mp4

$t \approx 0$

initialization
callback



init.mp4

再生を始めるための設定

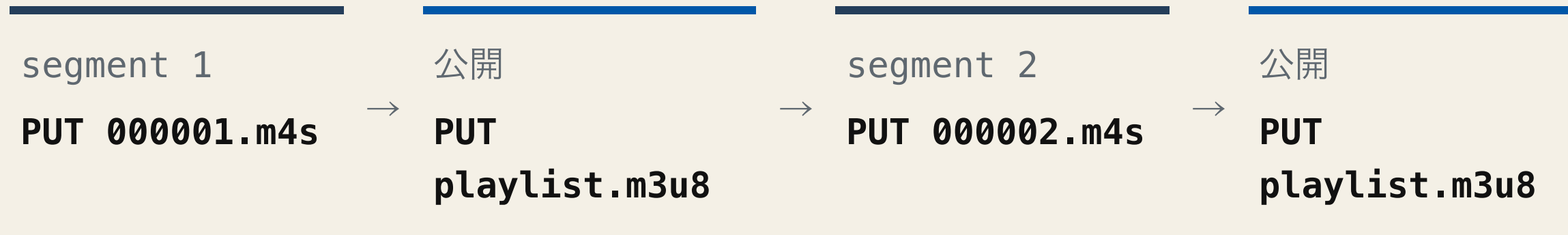


PUT

/streams/{id}/init.mp4

media segmentより先に、再生の前提を保存する

以後は約2秒ごとに、2つのPUTが続く



playlistが指すのは、保存に成功したsegmentだけ

Viewerは最新streamを見つけ、playlistを追いかける

1秒ごと

GET /api/streams



最新を選択

playlistURL



LIVE

Safari / AVPlayer /
hls.js

STREAM stream-... SEGMENTS 1 → 2 → 3 PLAYLIST /streams/.../playlist.m3u8

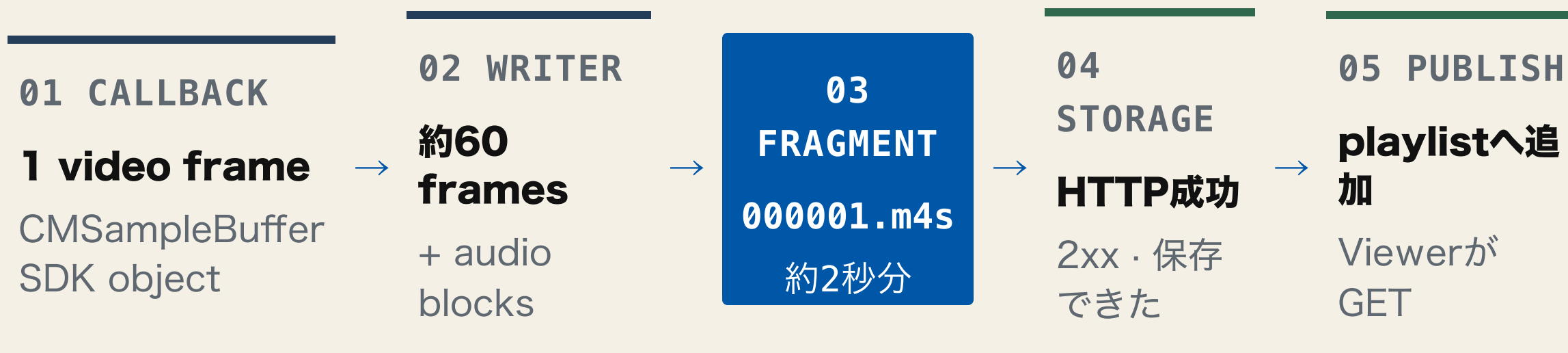
配信後に残る、3種類のHLSファイル

```
server/data/streams/stream-20260818-225315-C7A2AB11/
```

```
├─ init.mp4                                1,136 bytes
├─ playlist.m3u8                           682 bytes
└─ seg/                                    18 files
    000001.m4s 000002.m4s ... 000018.m4s
```

保存先で `ffplay playlist.m3u8` → HLS一式をそのまま再生確認

1 frameは、約2秒のsegmentへ まとめてから届ける



frame単体は送らない。約2秒分を保存してから、playlistで見えるようにする

02

Camera / Micから、 fMP4 Dataを取り出す

Camera / Mic → 1 frameずつ受け取る → Writer → HLS用Data

AVAssetWriterはsegmentを作る。playlistは作らない。

AVFoundation

圧縮して約2秒に分割

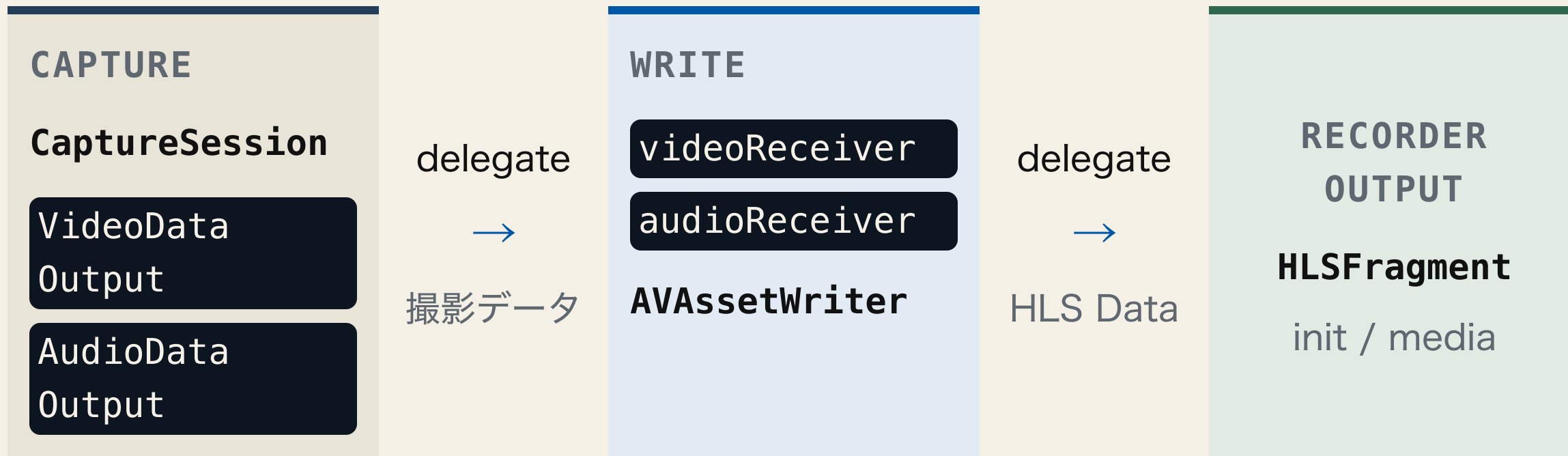
- 映像はH.264
- 音声はAAC
- init.mp4 + media segment

App

保存してから公開

- 保存先のpathを決める
- HTTP成功を確認する
- playlistへ追加する

6つのSDK objectを、2つのdelegateで接続



撮影データを、HLSとして保存できるDataへ変える

カメラ構成前に、録音用AudioSessionを有効化

```
1 let audio = AVAudioSession.sharedInstance()
2 try audio.setCategory(
3     .playAndRecord,
4     mode: .videoRecording,
5     options: [.defaultToSpeaker, .allowBluetoothHFP]
6 )
7 try audio.setActive(true)
```

Capture

camera + microphone

Route

speaker + Bluetooth HFP

CaptureSessionが、Camera / Micを DataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

video CMSampleBuffer

AudioDataOutput

audio CMSampleBuffer

完成した録画ファイルではなく、撮影中のframe / audio blockを受け取る

— ONE CAPTURE, THREE OUTPUTS

1つのCaptureSessionを、3つの用途で使う



`recording.mp4` → 元動画としてupload + 設定時は写真ライブラリへ保存

公開サンプルはPreview + HLS。MomentNowは保存用Writerも並行する

映像と音声を、同じqueueで受け取る

```
1 videoOutput.setSampleBufferDelegate(  
2     self, queue: writingQueue)  
3 audioOutput.setSampleBufferDelegate(  
4     self, queue: writingQueue)
```

VideoDataOutput

約1 frame

AudioDataOutput

短いaudio block

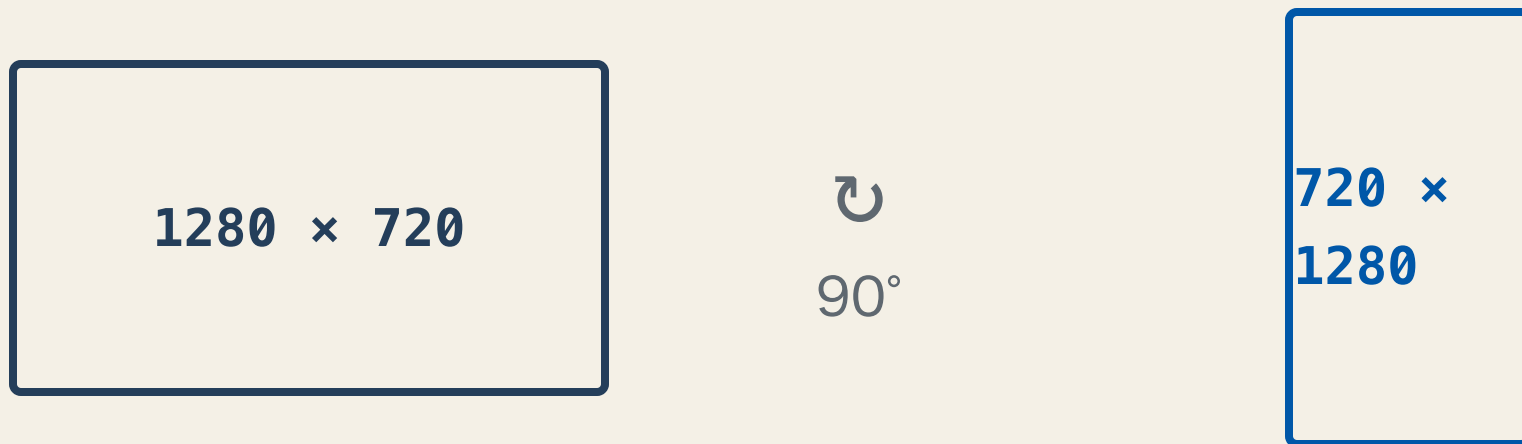


serial writingQueue

VideoとAudioを直列化し、Writerの状態を1か所で更新する

— ORIENTATION

縦向きは、video connectionへ90度を指定



```
1 if let connection = videoOutput.connection(with: .video),
2   connection.isVideoRotationAngleSupported(90) {
3   connection.videoRotationAngle = 90
4 }
```

配信開始前に、上り回線に合う単一品質を選ぶ

```
1 let profile: (CGSize, Int) = switch quality {  
2 case .high:    (.init(width: 720, height: 1280), 2_500_000)  
3 case .medium:  (.init(width: 720, height: 1280), 1_500_000)  
4 case .low:     (.init(width: 480, height: 854),    900_000)  
5 }
```

network状態

constrained /
expensive

API応答

配信準備の待ち時間

test upload

上り速度

2.0 sec

segment target

開始前に1品質を選ぶ。配信途中の自動画質切替（ABR）は行わない

未圧縮映像を、WriterでH.264へ圧縮

Capture

未圧縮映像

video frame

High H.264の圧縮profile名

append

Writer

H.264 High

圧縮された映像

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

サンプルは1.5 Mbps・個人アプリは上り回線に合わせて0.9-2.5 Mbps

配信用videoフォーマットは、互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別に選択
AverageBitRate	0.9–2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [  
2    AVFormatIDKey: kAudioFormatMPEG4AAC,  
3    AVNumberOfChannelsKey: 1,  
4    AVSampleRateKey: 44_100,  
5    AVEncoderBitRateKey: config.audioBitrate  
6  ]
```


HLS用のAVAssetWriterを構成する

```
1 let writer = AVAssetWriter(contentType: .mpeg4Movie)
2 writer.outputFileTypeProfile = .mpeg4AppleHLS
3 writer.preferredOutputSegmentInterval = .init(
4     seconds: 2, preferredTimescale: 600
5 )
6 writer.initialSegmentStartTime = startTimeOffset
7 writer.delegate = self
```

AVAssetWriter → delegate → initialization Data | separable Data

URLへ書き込まず、delegateへsegment Dataを出す

CMSampleBufferの入口はReceiver

```
1 let videoInput = AVAssetWriterInput(  
2     mediaType: .video,  
3     outputSettings: videoSettings()  
4 )  
5 self.videoReceiver =  
6     writer.inputReceiver(  
7         for: videoInput)
```

Video / H.264

videoReceiver

Audio / AAC

audioReceiver

↓ Inputを接続 ↓

AVAssetWriter

Videoを代表例として表示。Audioも同じ手順でReceiverを保持する

— CMSAMPLEBUFFER ≠ SEGMENT

多数のCMSampleBufferを、約2秒のsegmentへまとめる

Camera

video CMSampleBuffer

約1 frame · 30fpsなら約
33ms

Microphone

audio CMSampleBuffer

短い音声block



AVAssetWriter

圧縮

+ 時間順に格納
+ 約2秒に分割



1 配信に1つ

initialization

圧縮方式 · 映像/音声の構成

約2秒ごと

separable

video frame + audio block
本体

入力は小さなbuffer。出力は約2秒分の映像・音声

Writerへ渡す前に、2つのguardで確認する

captureOutput

配信中か
isWriting

Dataはreadyか
利用可能か確認

Writer開始
→ append

```
1 guard isWriting else { return }  
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

callbackごとに、Receiverへ1つつ渡す

DataOutput
callback



Writerへ渡す前
時刻を補正



CoreMedia
Readyなbuffer



Receiver
append

```
1 let readySampleBuffer = CMReadySampleBuffer(  
2     unsafeBuffer: transferableSampleBuffer  
3 )  
4 let didAppend = try receiver.appendImmediately(  
5     readySampleBuffer  
6 )  
7 guard didAppend else { return }
```

true Writerへ渡せた

false 待たずにdrop

throw streamをerror終了

WriterのDataを、Publisherが保存する

`.initialization` → init Data

→ `init.mp4`

`.separable` → media Data

→ `seg/000001.m4s`

```
1 switch segmentType {
2 case .initialization:
3     continuation.yield(.initialization(segmentData))
4 case .separable:
5     segmentIndex += 1
6     continuation.yield(.media(sequence: segmentIndex,
7         data: segmentData, duration: duration))
8 }
```

Recorderの出力はData。HTTPHLSClientが保存先のpathを組み立てる

HLS生成の8ステップ

準備

- 01 Captureを構成
Camera / MicをSessionへ追加
- 02 DataOutputを接続
delegateとserial queueを指定
- 03 Writerを構成
HLS profileと約2秒の設定
- 04 Receiverを保持
Video / Audio Inputを接続

実行・停止

- 05 最初のVideoで開始
`start()` → `startSession`
- 06 時刻を補正してappend
`appendImmediately`
- 07 delegateでDataを受信
initialization → separable
- 08 停止時にflush
`finishWriting`で最後のData

05・06では、最初の映像を基準にWriterへ渡す時刻を調整する

Writerへ渡す時刻の調整

時刻の扱いは、Apple公式サンプルの方針を採用

最初の映像を10秒に置き、映像と音声を同じ量だけずらす

同じ90秒を引くので、時間差は変わらない

	調整前の時刻	Writerへ渡す時刻
映像	100.00 秒	10.00 秒
音声	100.02 秒	10.02 秒

検証結果：時刻調整を外すと、映像・音声が入らなかった

**検証条件：Writerの開始位置は10秒のまま
時刻調整だけを外した場合**

03

映像と音声を、 約2秒のfMP4へ分ける

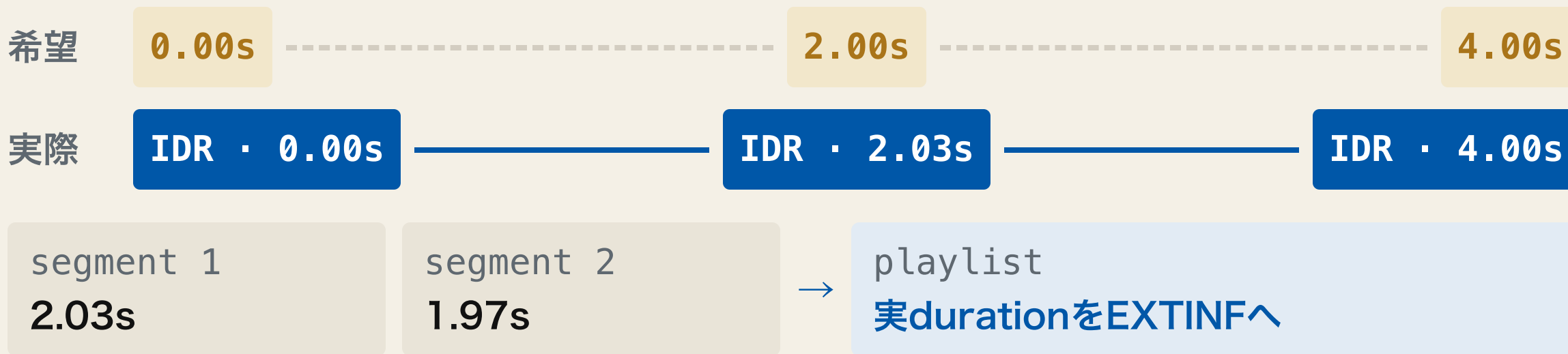
Four HLS settings and segment boundaries

今回のHLS Writerで使う、4つの設定

- | | | |
|----|---|-----------------|
| 01 | <code>outputFileTypeProfile</code> | Apple HLS向けfMP4 |
| 02 | <code>preferredOutputSegmentInterval</code> | 希望するsegment間隔 |
| 03 | <code>initialSegmentStartTime</code> | 最初のsegment開始時刻 |
| 04 | <code>delegate</code> | 生成されたDataの受け口 |

— PREFERRED INTERVAL

2秒ぴったりではなく、2秒付近のIDRで切る



IDR ほかのframeを参照せず、そこから単独で再生を始められるkeyframe

2秒は設定値。playlistにはAVAssetSegmentReportの実durationを書く

実durationの反映：サンプルで実装済み

公開サンプル ・ 実装済み

実durationをEXTINFへ

reportが無効・未取得の場合だけ2秒

MomentNow ・ 現状

2秒固定で送信

改善案：同じ実durationの取り込みを適用

```
1 let duration = report?.trackReports
2   .first { $0.mediaType == .video }?
3   .duration.seconds
4 guard let duration, duration.isFinite, duration > 0 else {
5   return config.segmentSeconds
6 }
7 return duration
```

— KEYFRAME ALIGNMENT

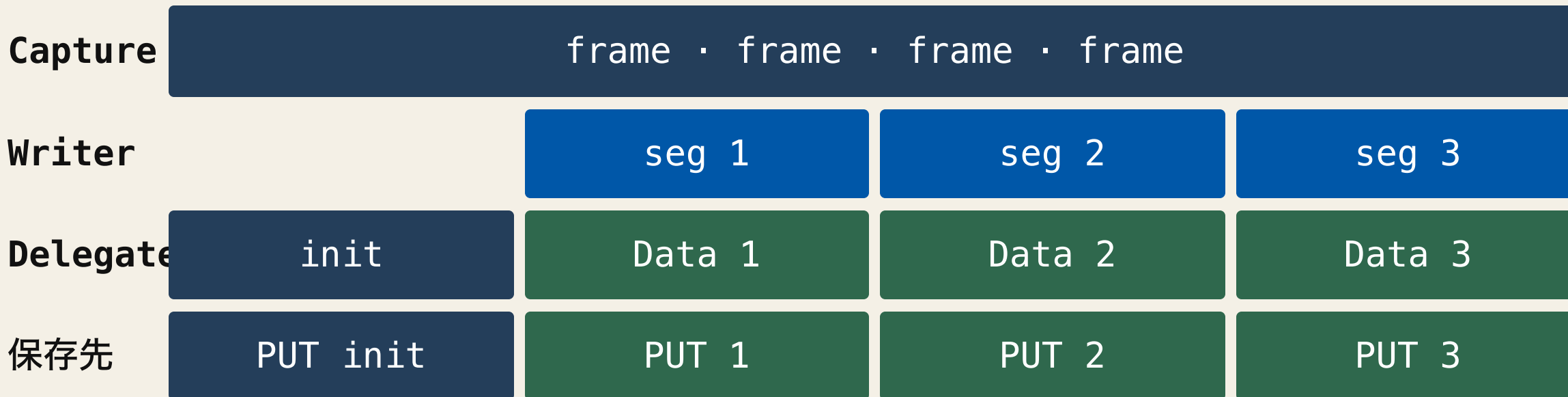
Appleは、約2秒ごとのIDRを推奨



1 `AVVideoMaxKeyFrameIntervalDurationKey`: `config.segmentSeconds`

IDR ほかのframeを参照せず、そこから再生を始められるkeyframe

Captureを止めず、約2秒ごとにDataを出す



Writerのfinishを待たず、生成と保存を並行できる

04

保存できたsegmentだけを、 playlistへ公開する

Save bytes first, then make the segment visible to viewers

— SAME HLS · DIFFERENT DESTINATION

同じHLSを、MacまたはS3へ置く

公開サンプ
ル

iPhone

HTTP PUT

Macのファイル保存

HTTP
GET

**ローカル
Viewer**

MomentNow

iPhone

署名付き
URLへPUT

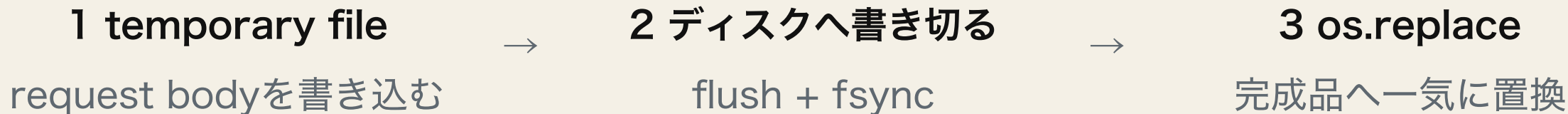
Amazon S3
object storage

CloudFront
CDN

Web Viewer

init.mp4とm4sのDataは、どちらもiPhoneが生成する

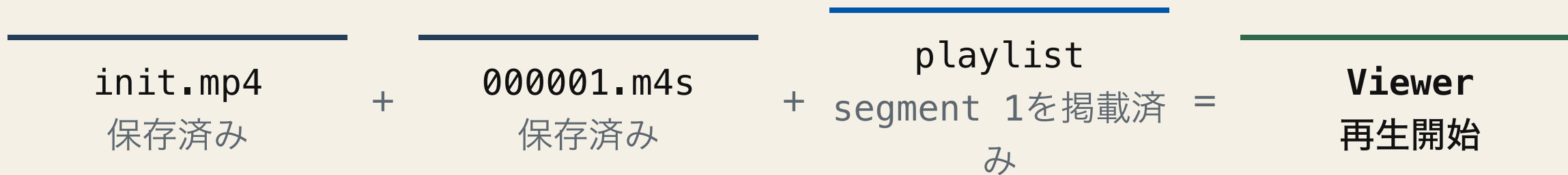
PUT中のファイルを、Viewerへ見せない



playlistの途中状態や、半分だけのm4sをGETさせない

再生開始は、init保存と最初のsegment公開の後

start	network	~2s	after PUT
init Data	PUT init	PUT seg 1	playlist更新



playlist公開後に、アプリ内の状態を確定する

```
1 var nextManifest = manifest
2 nextManifest.addSegment(seq: seq, durationSec: durationSec)
3
4 try await client.putPlaylist(
5     streamId: streamId, text: nextManifest.text)
6 manifest = nextManifest
```

current
segments 1...N

copy

candidate
+ segment N+1

HTTP
成功

状態を確定
current =
candidate

playlistのPUT成功後に、アプリ内の状態を確定する

署名付きURL → S3 PUT → playlist公開

APPの操作	MomentNow	結果
S3への署名付きURLを受け取る	POST /presign	presigned PUT URL
segment本体を保存する	PUT → S3	objectが存在する
playlistへ載せる	POST /commit	Viewerから見える

S3は保存先の選択。HLSで重要なのは「保存してから公開」

S3へのPUTを、署名付きURLで許可する

DO NOT

AWS credentialを内包

漏えい範囲が広い



PRESIGNED URL

S3へ1ファイルだけPUT

保存先 / 有効期限を署名

iPhoneは発行されたURLへDataをPUTするだけ。AWS credentialは持たない

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(
4     contentType, forHTTPHeaderField: "Content-Type")
5 request.httpBody = data
```

— INIT ONCE · MEDIA REPEATS

mediaは、PUT後にcommitを追加

```
1 let url = try await api.presign(kind: kind, seq: seq)
2 try await api.put(to: url, data: data)
```

INITIALIZATION · 1回

```
kind: "init", seq: nil
```

PUT成功で完了

MEDIA · 約2秒ごと

```
kind: "segment", seq: seq
```

PUT成功後に追加

```
try await api.commit(seq: seq)
```

mediaだけは、PUT成功後にcommitしてplaylistへ載せる

— PUBLISH AFTER BYTES

PUT成功より先に、playlistへseqを出さない

BAD

commit

→

Viewer GET

→

404

GOOD

S3 PUT完了

→

commit

→

playlist更新

サンプルは、同じファイルを 最大3回まで再送する

attempt 1

失敗

250 ms



attempt 2

失敗

500 ms



attempt 3

成功 / error

終了

segment PUTが失敗したら、そのsegmentをplaylistへ載せない

— OUT-OF-ORDER RACE

upload完了順と、再生順は一致しないことがある

時間 →

seg 1  commit 1

seg 2  commit 2

seg 3  commit 3

completion **2, 1, 3** ≠ playback **1, 2, 3**

MomentNowの現状 sequence順へ整列。欠番は待たない

改善案・未実装 欠番を待ち、連続した範囲だけ公開

cache方針は、playlistとsegmentで分ける

OBJECT	CACHE	HTTP
playlist.m3u8	no-store / no-cache	毎回最新を取得
init.mp4 / m4s	max-age=31536000	Range / 206（部分取得）

S3 / CloudFrontでの設定

05

生成を止めず、 配信の完了まで待つ

AVFoundation callback → Swift Task → final playlist

メディア処理を、HTTP待ちから切り離す

WRITING QUEUE ·
CALLBACK

WriterのDataを受け取る

init / mediaを受け取り
すぐ戻る

Recorder



BRIDGE

Dataを順番に受け渡す

callbackの値を
非同期の列へ変換

AsyncThrowing
Stream



SWIFT TASK

HTTPをawaitする

PUTとplaylist更新を
順番に待つ

Publisher

HLSFragment = init / mediaのData

送信待ちに上限なし。送信が遅いとsegmentが滞留する

Writerの詰まりと、送信待ちは別の問題

Capture	video 1	audio 1	video 2	audio 2
Writer	append	append	busy	drop

Writer側・実装済み

受け取れないCMSampleBufferをdrop
映像のカクつき・音声の欠けを許容

送信側・今後の改善

送信待ち量に上限を設ける
上限超過時の停止判断を追加

Writer側のdropでは、生成済みsegmentの送信待ちは減らない

— WRITER FINISH

最後に渡した時刻で、Writerを終了する

lastAdjustedPTS

最後にappendした時刻

endSession

→ media rangeを閉じる

markAsFinished

→ video / audio

finishWriting

→ 最後のsegmentをflush

```
1 if lastAdjustedPTS.isValid {  
2     writer.endSession(atSourceTime: lastAdjustedPTS)  
3 }
```

撮影停止後も、最後のsegment公開まで待つ

1 Recorder

capture停止 →
finishWriting

最後のfragmentが出る

2 Stream

→ continuation.finish()
fragment列を閉じる

3 Publisher

→ uploadTask.value
ENDLIST PUTまで待つ

「撮影停止」と「配信完了」は、同じ瞬間ではない

— PUBLIC DEMO → PRODUCTION

MomentNowで採用した4つの運用制御

AUTH

user / group
ownership

PRESIGN

S3の1ファイル限
定・署名付きURL

COMMIT

公開順とplaylistの
同時更新を制御

DELIVERY

CloudFront /
ticket / status

commit APIは必須ではない。サンプルはiOSでplaylistと公開順を管理する

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線



USE A MEDIA PLATFORM

大規模・長時間・複数画質の自動切替

厳しい可用性要件にも対応したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

端末内で生成したHLSを、撮影中から公開する

01 生成 ・ Writer

Camera / Micの入力からHLS用Dataを作る

02 区切り ・ Boundary

IDRとsegment intervalをそろえる

03 公開 ・ Upload

Dataを保存してからplaylistへ載せる

04 終了 ・ Finish

全送信とENDLISTの公開を待つ

端末内生成 AVCaptureSession → DataOutput → Receiver → AVAssetWriterDelegate

ありがとうございました

サンプルアプリのコード

<https://github.com/HikaruSato/iosdc2026HLSSample>

— APPENDIX

補足：Writerの時刻処理

開始の基準 / 10秒の理由 / 時刻を変えるコード

最初の映像を基準にWriterを開始

先に届いた音声

まだappendしない

最初の映像

`writer.start()`

10秒を開始位置に

映像・音声に共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
3 try writer.start()
4 let pts = CMSampleBufferGetPresentationTimeStamp(sampleBuffer)
5 writer.startSession(atSourceTime: startTimeOffset)
6 timeOffsetDelta = startTimeOffset - pts
```

10秒は、音声圧縮のための余白

- AACは圧縮の都合で、先頭に準備用の音声（priming）を加える
- Apple HLSでは、その分だけ音声の時刻を前へずらして補償する

映像・音声の開始位置を後ろへずらし、
音声の時刻が負にならないようにする

```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)  
2 writer.initialSegmentStartTime = startTimeOffset
```

10秒はAppleが示す設定例。再生開始まで10秒待つ意味ではない

— APPENDIX · ONE DELTA

時刻補正は、全サンプルの平行移動

$$\text{delta} = 10s - \text{firstVideoPTS}$$

$$\text{adjustedPTS} = \text{sourcePTS} + \text{delta}$$

PTS：映像や音声を提示する時刻。移動量は最初の映像で一度だけ決める

映像・音声は変えず、時刻情報だけを補正

```
1 let timingInfos = try sampleTimingInfos().map { info in
2     var t = info
3     t.presentationTimeStamp = t.presentationTimeStamp + offset
4     if info.decodeTimeStamp.isValid {
5         t.decodeTimeStamp = t.decodeTimeStamp + offset
6     }
7     return t
8 }
9 let copied = try CMSampleBuffer(
10     copying: self, withNewTiming: timingInfos)
```

PTSと、有効なDTSを同じ量だけ動かす